Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий
Кафедра прикладной математики

Отчет защищен с оценкой _____

Преподаватель _____
(подпись)
«___» _____ 2026 г.

Отчет
По лабораторной работе №3

«Структуры»

по дисциплине «Программирование – 2 семестр»

Студент группы ПИ-54 Дворников М. С.

Преподаватель доцент,к.т.н. Егорова Е.В.

Барнаул 2026

Задание 1. (№9). Написать программу по обработке массива структур
Имеется таблица “Автомобили”.
Каждая строка содержит следующие сведения:
марка, номер, фамилия владельца, технические характеристики (не более 3), каждая характеристика - целое число.
Упорядочить таблицу по фамилиям владельцев и найти фамилию владельца и номера автомобилей заданной марки.

Текст программы:

```
// Создание структуры
typedef struct {
    char brand[TEXT_LEN + 1];  // Марка автомобиля
    char plate[TEXT_LEN + 1];  // Номерной знак
    char surname[TEXT_LEN + 1];  // Фамилия владельца
    // Тех. характеристики
    int power, weight, capacity;  // Мощность (в л.с.), вес и вместимость (чел. в салоне)
} Car;


// Ввод таблицы "Автомобили"
int table_input(Car** pointer) {
    // pointer: указатель на массив (таблицу)
    // Возвращает количество строк в таблице

    int n = 0;  // Кол-во строк в таблице а так же номер "пустого" индекса
    int to_end = 0;  // Нужно закончить ввод
    while (n < ROWS && !to_end) {
        printf("Сейчас в таблице строк: %d. Чтобы закончить ввод, нажмите ESC.\n"
            "Нажмите любую другю кнопку, чтобы ввести %d строку:\n", n, n + 1);
        char button = _getch();
        if (button == 27) {  // Нажат ESC
            if (n > 0) to_end = 1;
            else printf("\nНельзя закончить ввод, если таблица пустая.\n");
        }
        else {  // Ввод таблицы по столбцам (т.е. вводим структуры)
            // Ввод полей "марка", "номер", "фамилия младельца"
            const char* names[3];                        char* char_fields[3];
            names[0] = "\nВведите марку автомобиля";    char_fields[0] = (*pointer)[n].brand;
            names[1] = "\nВведите номер автомобиля";    char_fields[1] = (*pointer)[n].plate;
            names[2] = "\nВведите фамилию владельца";   char_fields[2] =
(*pointer)[n].surname;
            for (int col = 0; col < 3; col++) {
                printf(names[col]);
                printf("(не более % d символов):\n", TEXT_LEN);  // Заканчиваем сообщение
                char user_input[TEXT_LEN + 1];  // Создаём буфер для ввода
                do {
                    fgets(user_input, TEXT_LEN + 1, stdin);
                    int len = strlen(user_input);
                    if (user_input[len - 1] != '\n') while (getchar() != '\n');  // Очищаем
входной поток
                    else user_input[len - 1] = '\0';
                } while (strlen(user_input) == 0);
                strcpy(char_fields[col], user_input);  // Сохраняем строку в нужное поле
            }
            // Ввод мощности
            printf("\nВведите мощность (>0) автомобиля в л.с.:\n");
            do scanf("%d", &(*pointer)[n].power);
            while ((*pointer)[n].power <= 0);
            // Ввод веса
            printf("\nВведите вес (>0) автомобиля в кг.:\n");
            do scanf("%d", &(*pointer)[n].weight);
            while ((*pointer)[n].weight <= 0);
```

```
            // Ввод вместиморсти
            printf("\nВведите вместимость чел. (0< <9) автомобиля:\n");
            do scanf("%d", &(*pointer)[n].capacity);
            while ((*pointer)[n].capacity <= 0 || 9 <= (*pointer)[n].capacity);

            printf("\nСтрока %d успешно записана!\n", n + 1);
            n++;
        }
    }
    return n;
}

// Нарисовать грань таблицы
void draw_edge() {
    const char* edge = "-+|";  // Границы таблицы
    printf("%c", edge[1]);
    for (int i = 1; i < COLUMNS_NUM * (TEXT_LEN + 1); i++)  // Рисуем "+---+---+" - границу
таблицы
        if (i % (TEXT_LEN + 1)) printf("%c", edge[0]);
        else printf("%c", edge[1]);  // Через каждые TEXT_LEN символов ставим "+"
    printf("%c\n", edge[1]);
}

// Вывод таблицы "Автомобили"
void table_output(Car* table, int len) {
    // pointer: указатель на массив (таблицу)
    // len: количество строк таблицы

    const char* edge = "-+|";  // Границы таблицы
    const char* head[2][COLUMNS_NUM];  // Шапка таблицы
    head[0][0] = "Марка";       head[1][0] = "автомобиля";
    head[0][1] = " ";           head[1][1] = "Номер";
    head[0][2] = "Фамилия";     head[1][2] = "владельца";
    head[0][3] = "Мощность";    head[1][3] = "(л.с.)";
    head[0][4] = "Вес";         head[1][4] = "(кг.)";
    head[0][5] = "Вместимость"; head[1][5] = "(чел.)";
    draw_edge();  // Верхняя граница таблицы
    // Шапка таблицы
    for (int i = 0; i < 2; i++) {
        printf("%c", edge[2]);
        for (int w = 0; w < COLUMNS_NUM; w++) {
            const char* cur_word = head[i][w];      // Текущее слово
            int word_len = strlen(cur_word);  // Его длина
            int left_space = (TEXT_LEN - word_len) / 2;
            for (int i = 0; i < left_space; i++) printf(" ");
            // Вывод слова
            printf("%s", cur_word);
            // Расстояние справа
            int right_space = TEXT_LEN - word_len - left_space;
            for (int i = 0; i < right_space; i++) printf(" ");
            printf("%c", edge[2]);
        }
        printf("\n");
    }
    // Информация в таблице
    for (int i = 0; i < len; i++) {
        draw_edge();  // Граница таблицы
        // Строчка
        printf("%c", edge[2]);
        // Вывод текстовых полей
        char* char_fields[] = { table[i].brand, table[i].plate, table[i].surname };
        for (int w = 0; w < 3; w++) {
            const char* cur_word = char_fields[w];      // Текущее слово
            int word_len = strlen(cur_word);  // Его длина
            int left_space = (TEXT_LEN - word_len) / 2;
            for (int i = 0; i < left_space; i++) printf(" ");
```

```
            // Вывод слова
            printf("%s", cur_word);
            // Расстояние справа
            int right_space = TEXT_LEN - word_len - left_space;
            for (int i = 0; i < right_space; i++) printf(" ");
            printf("%c", edge[2]);
        }
        // Вывод числовых полей
        int int_fields[] = { table[i].power, table[i].weight, table[i].capacity };
        for (int w = 0; w < 3; w++) {
            int num_len = log10(int_fields[w]) + 1;  // Его длина
            int left_space = (TEXT_LEN - num_len) / 2;
            for (int i = 0; i < left_space; i++) printf(" ");
            // Вывод слова
            printf("%d", int_fields[w]);
            // Расстояние справа
            int right_space = TEXT_LEN - num_len - left_space;
            for (int i = 0; i < right_space; i++) printf(" ");
            printf("%c", edge[2]);
        }
        printf("\n");
    }
    draw_edge();  // Нижняя граница таблицы
}


int main() {
    SETIO;
    system("color F0");
    printf("Программа для таблицы \"Автомобили\"\n");
    Car table[ROWS];  // Таблица с автомобилями
    Car* table_pointer = table;  // Указатель на массив (таблицу)
    // Заполнение таблицы
    int len = table_input(&table_pointer);
    printf("\nНажмите любую кнопку, чтобы продолжить\n");
    _getch();
    do {
        system("cls");
        // Вывод таблицы
        printf("Получилась таблица:\n");
        table_output(table, len);
        // Получение и вывод сортированной по фамилиям таблицы
        Car sorted_table[ROWS];  // Копия таблицы
        for (int i = 0; i < len; i++) sorted_table[i] = table[i];
        // Сортировка копии таблицы
        for (int k = 1; k < len; k++)
            for (int i = 0; i < len - k; i++)
                if (strcmp(sorted_table[i].surname, sorted_table[i + 1].surname) > 0) {
                    Car buffer = sorted_table[i];
                    sorted_table[i] = sorted_table[i + 1];
                    sorted_table[i + 1] = buffer;
                }
        printf("\nСортированная по фамиилиям владельцев таблица:\n");
        table_output(sorted_table, len);
        // Ввод строки
        printf("\nДоступен запрос \"поиск по маркам\". Введите нужную марку:\n");
        char need_brand[TEXT_LEN + 1];  // Создаём буфер для ввода
        do {
            fgets(need_brand, TEXT_LEN + 1, stdin);
            int brand_len = strlen(need_brand);
            if (need_brand[brand_len - 1] != '\n') while (getchar() != '\n');  // Очищаем
входной поток
            else need_brand[brand_len - 1] = '\0';
        } while (strlen(need_brand) == 0);
        int flag = 0;  // Найдена хоть одна машина нужной марки
        for (int i = 0; i < len; i++)
```

```
            if (!strcmp(sorted_table[i].brand, need_brand)) {
                printf("\nНайдена машина марки %s: владелец - %s, номер - \"%s\"",
                    need_brand, sorted_table[i].surname, sorted_table[i].plate);
                flag = 1;
            }
        if (!flag) printf("\nМашин данной марки нет в таблице.");
        printf("\n\nНажмите ESC, что бы закончить.\nНажмите любую другую кнопку, чтобы сделать
ещё один запрос:\n");
    } while (_getch() != 27);
    return 0;
}
```

Тесты программы:

В самом начале программа выдаёт:

```
Программа для таблицы "Автомобили"
Сейчас в таблице строк: 0. Чтобы закончить ввод, нажмите ESC.
Нажмите любую другю кнопку, чтобы ввести 1 строку:
```

Нельзя закончить ввод, если в таблице нет строк:

```
Нельзя закончить ввод, если таблица пустая.
Сейчас в таблице строк: 0. Чтобы закончить ввод, нажмите ESC.
Нажмите любую другю кнопку, чтобы ввести 1 строку:
```

Пустой или неправильный ввод не засчитывается:

```
Введите марку автомобиля(не более  15 символов):


▬
```

```
Введите вместимость чел. (0< <9) автомобиля:
10
▬
```

Если завершить ввод с непустой таблицей, то ввод кончится:

```
Нажмите любую кнопку, чтобы продолжить
```

Введём произвольные данные:

```
Получилась таблица:
+---------------+---------------+---------------+---------------+---------------+---------------+
|     Марка     |               |    Фамилия    |   Мощность    |      Вес      |  Вместимость  |
|  автомобиля   |     Номер     |   владельца   |    (л.с.)     |     (кг.)     |    (чел.)     |
+---------------+---------------+---------------+---------------+---------------+---------------+
|    Toyota     |    812мсд     |    Суровец    |      300      |     1000      |       4       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|    Toyota     |    128мсд     |   Дворников   |      600      |     1100      |       2       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|     Lexus     |    1488хай    |  Лавриненко   |      450      |     1500      |       5       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|   Mercedes    |    п000си     |   Балицкий    |      400      |     1200      |       4       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|      BMW      |       -       |    Елунин     |      900      |      900      |       2       |
+---------------+---------------+---------------+---------------+---------------+---------------+

Сортированная по фамиилиям владельцев таблица:
+---------------+---------------+---------------+---------------+---------------+---------------+
|     Марка     |               |    Фамилия    |   Мощность    |      Вес      |  Вместимость  |
|  автомобиля   |     Номер     |   владельца   |    (л.с.)     |     (кг.)     |    (чел.)     |
+---------------+---------------+---------------+---------------+---------------+---------------+
|   Mercedes    |    п000си     |   Балицкий    |      400      |     1200      |       4       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|    Toyota     |    128мсд     |   Дворников   |      600      |     1100      |       2       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|      BMW      |       -       |    Елунин     |      900      |      900      |       2       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|     Lexus     |    1488хай    |  Лавриненко   |      450      |     1500      |       5       |
+---------------+---------------+---------------+---------------+---------------+---------------+
|    Toyota     |    812мсд     |    Суровец    |      300      |     1000      |       4       |
+---------------+---------------+---------------+---------------+---------------+---------------+

Доступен запрос "поиск по маркам". Введите нужную марку:
```

Проверим запросы на 2, 1 и 0 результатов:

```
Доступен запрос "поиск по маркам". Введите нужную марку:
Toyota

Найдена машина марки Toyota: владелец - Дворников, номер - "128мсд"
Найдена машина марки Toyota: владелец - Суровец, номер - "812мсд"
```

```
Доступен запрос "поиск по маркам". Введите нужную марку:
Lexus

Найдена машина марки Lexus: владелец - Лавриненко, номер - "1488хай"
```

```
Доступен запрос "поиск по маркам". Введите нужную марку:
Я устал

Машин данной марки нет в таблице.
```

Задание 2. (№9). Написать программу по обработке массива переменных структур

Имеется телефонный справочник.
Если телефон личный, то строка справочника содержит: 1) номер телефона, 2) фамилию владельца, 3) адрес владельца;
Если телефон служебный - 1) номер телефона, 2) наименование учреждения, 3) номер отдела.
Определить все номера телефонов, владельцы которых имеют заданную фамилию, и упорядочить их по адресам.

Текст программы:

```
// Создание структуры
typedef struct {
    char phone[TEXT_LEN + 1];  // Номер телефона
    char type;  // Личный (л) или служебный (с)
    union {  // Переменная информация
        struct {  // Информация для личных телефонов
            char surname[TEXT_LEN + 1];  // Фамилия владельца
            char address[TEXT_LEN + 1];  // Адрес владельца
        } personal;
        struct {  // Информация для служебных телефонов
            char office[TEXT_LEN + 1];  // Наименование учреждения
            char num[TEXT_LEN + 1];  // Номер отдела
        } servise;

    }info;
} Directory;


// Ввод телефонного справочника
int table_input(Directory** pointer) {
    // pointer: указатель на массив (таблицу)
    // Возвращает количество строк в таблице

    int n = 0;  // Кол-во строк в таблице а так же номер "пустого" индекса
    int to_end = 0;  // Нужно закончить ввод
    while (n < ROWS && !to_end) {
        printf("Сейчас в таблице строк: %d. Чтобы закончить ввод, нажмите ESC.\n"
            "Нажмите любую другю кнопку, чтобы ввести %d строку:\n", n, n + 1);
        char button = _getch();
        if (button == 27) {  // Нажат ESC
            if (n > 0) to_end = 1;
            else printf("\nНельзя закончить ввод, если таблица пустая.\n");
        }
```

```
        else {  // Ввод таблицы по столбцам (т.е. вводим структуры)
            // Ввод типа телефона (личный или служебный)
            printf("\nКакой это телефон: личный (введите \"л\") или служебный (\"с\")?\n");
            do scanf("%c", &(*pointer)[n].type);
            while ((*pointer)[n].type != 'л' && (*pointer)[n].type != 'с');
            int is_servise = (*pointer)[n].type == 'с';  // 1, если телефон личный, и 0, если
служебный
            // Ввод полей
            const char* names[2][3];
            names[0][0] = names[1][0] = "\nВведите номер телефона";
            names[0][1] = "\nВведите фамилию владельца";    names[1][1] = "\nВведите
наименование учреждения";
            names[0][2] = "\nВведите адрес владельца";      names[1][2] = "\nВведите номер
отдела";
            char* char_fields[2][3];
            char_fields[0][0] = char_fields[1][0] = (*pointer)[n].phone;
            char_fields[0][1] = (*pointer)[n].info.personal.surname;    char_fields[1][1] =
(*pointer)[n].info.servise.office;
            char_fields[0][2] = (*pointer)[n].info.personal.address;    char_fields[1][2] =
(*pointer)[n].info.servise.num;

            for (int col = 0; col < 3; col++) {
                printf(names[is_servise][col]);
                printf(" (не более % d символов):\n", TEXT_LEN);  // Заканчиваем сообщение
                char user_input[TEXT_LEN + 1];  // Создаём буфер для ввода
                do {
                    fgets(user_input, TEXT_LEN + 1, stdin);
                    int len = strlen(user_input);
                    if (user_input[len - 1] != '\n') while (getchar() != '\n');  // Очищаем
входной поток
                    else user_input[len - 1] = '\0';
                } while (strlen(user_input) == 0);
                strcpy(char_fields[is_servise][col], user_input);  // Сохраняем строку в
нужное поле
            }
            printf("\nСтрока %d успешно записана!\n", n + 1);
            n++;
        }
    }
    return n;
}

// Нарисовать грань таблицы
void draw_edge() {
    const char* edge = "-+|";  // Границы таблицы
    printf("%c", edge[1]);
    for (int i = 1; i < COLUMNS_NUM * (TEXT_LEN + 1); i++)  // Рисуем "+---+---+" - границу
таблицы
        if (i % (TEXT_LEN + 1)) printf("%c", edge[0]);
        else printf("%c", edge[1]);  // Через каждые TEXT_LEN символов ставим "+"
    printf("%c\n", edge[1]);
}

// Вывод телефонного справочника
void table_output(Directory* table, int len) {
    // pointer: указатель на массив (таблицу)
    // len: количество строк таблицы

    const char* edge = "-+|";  // Границы таблицы
    const char* head[2][COLUMNS_NUM];  // Шапка таблицы
    head[0][0] = "Тип";              head[1][0] = "телефона";
    head[0][1] = " ";                head[1][1] = "Номер";
    head[0][2] = "Фамилия влад./";   head[1][2] = "наименование уч";
    head[0][3] = "адрес влад./";     head[1][3] = "номер отдела";
    draw_edge();  // Верхняя граница таблицы
```

```
    // Шапка таблицы
    for (int i = 0; i < 2; i++) {
        printf("%c", edge[2]);
        for (int w = 0; w < COLUMNS_NUM; w++) {
            const char* cur_word = head[i][w];      // Текущее слово
            int word_len = strlen(cur_word);  // Его длина
            int left_space = (TEXT_LEN - word_len) / 2;
            for (int i = 0; i < left_space; i++) printf(" ");
            // Вывод слова
            printf("%s", cur_word);
            // Расстояние справа
            int right_space = TEXT_LEN - word_len - left_space;
            for (int i = 0; i < right_space; i++) printf(" ");
            printf("%c", edge[2]);
        }
        printf("\n");
    }
    // Информация в таблице
    for (int i = 0; i < len; i++) {
        draw_edge();  // Граница таблицы
        // Вывод текстовых полей
        printf("%c%*c%*c", edge[2],
            TEXT_LEN / 2 + 1, table[i].type,
            TEXT_LEN / 2 + 1, edge[2]);  // Вывод типа телефона
        char* char_fields[COLUMNS_NUM - 1];
        char_fields[0] = table[i].phone;
        if (table[i].type == 'л') {  // Если это личный телефон
            char_fields[1] = table[i].info.personal.surname;
            char_fields[2] = table[i].info.personal.address;
        }
        else {
            char_fields[1] = table[i].info.servise.office;
            char_fields[2] = table[i].info.servise.num;
        }

        for (int w = 0; w < COLUMNS_NUM - 1; w++) {
            const char* cur_word = char_fields[w];      // Текущее слово
            int word_len = strlen(cur_word);  // Его длина
            int left_space = (TEXT_LEN - word_len) / 2;
            for (int i = 0; i < left_space; i++) printf(" ");
            // Вывод слова
            printf("%s", cur_word);
            // Расстояние справа
            int right_space = TEXT_LEN - word_len - left_space;
            for (int i = 0; i < right_space; i++) printf(" ");
            printf("%c", edge[2]);
        }
        printf("\n");
    }
    draw_edge();  // Нижняя граница таблицы
}


int main() {
    SETIO;
    system("color F0");
    printf("Программа для телефонного справочника\n");
    Directory table[ROWS];  // Телефонный справочник
    Directory* table_pointer = table;  // Указатель на массив (справочник)
    // Заполнение справочника
    int len = table_input(&table_pointer);
    // Получение сортированного по адресам справочника только с личными номерами
    Directory sorted_personal_table[ROWS];  // Копия справочника только с личными номерами
    int n = 0;  // Длина нового справочника
    for (int i = 0; i < len; i++)
        if (table[i].type == 'л')
```

```
            sorted_personal_table[n++] = table[i];
    // Сортировка копии справочника
    for (int k = 1; k < n; k++)
        for (int i = 0; i < n - k; i++)
            if (strcmp(sorted_personal_table[i].info.personal.address, sorted_personal_table[i
+ 1].info.personal.address) > 0) {
                Directory buffer = sorted_personal_table[i];
                sorted_personal_table[i] = sorted_personal_table[i + 1];
                sorted_personal_table[i + 1] = buffer;
            }
    printf("\nНажмите любую кнопку, чтобы продолжить\n");
    _getch();
    do {
        system("cls");
        // Вывод таблицы
        printf("Получился справочник:\n");
        table_output(table, len);
        printf("После сортировки получился такой справочник:\n");
        table_output(sorted_personal_table, n);
        // Ввод строки
        printf("\nДоступен запрос \"поиск по фамилиям\". Введите нужную фамилию:\n");
        char need_surname[TEXT_LEN + 1];  // Создаём буфер для ввода
        do {
            fgets(need_surname, TEXT_LEN + 1, stdin);
            int surname_len = strlen(need_surname);
            if (need_surname[surname_len - 1] != '\n') while (getchar() != '\n');  // Очищаем
входной поток
            else need_surname[surname_len - 1] = '\0';
        } while (strlen(need_surname) == 0);
        int flag = 0;  // Найден хоть один номер
        for (int i = 0; i < len; i++)
            if (!strcmp(sorted_personal_table[i].info.personal.surname, need_surname)) {
                printf("\nПо фамилии \"%s\" найден номер: %s",
                    need_surname, sorted_personal_table[i].phone);
                flag = 1;
            }
        if (!flag) printf("\nВ справочнике нет номеров, принадлежащих этому человеку");
        printf("\n\nНажмите ESC, что бы закончить.\nНажмите любую другую кнопку, чтобы сделать
ещё один запрос:\n");
    } while (_getch() != 27);
    return 0;
}
```

Тесты программы:

В самом начале программа выдаёт:

```
Программа для телефонного справочника
Сейчас в таблице строк: 0. Чтобы закончить ввод, нажмите ESC.
Нажмите любую другю кнопку, чтобы ввести 1 строку:
```

Нельзя закончить ввод, если в таблице нет строк:

```
Нельзя закончить ввод, если таблица пустая.
Сейчас в таблице строк: 0. Чтобы закончить ввод, нажмите ESC.
Нажмите любую другю кнопку, чтобы ввести 1 строку:
```

Пустой или неправильный ввод не засчитывается:

```
Введите номер телефона (не более  15 символов):


Какой это телефон: личный (введите "л") или служебный ("с")?
ы
```

Если завершить ввод с непустой таблицей, то ввод кончится:

```
Нажмите любую кнопку, чтобы продолжить
```

Введём произвольные данные:

```
Получился справочник:
+---------------+---------------+---------------+---------------+
|      Тип      |               |Фамилия влад./ | адрес влад./  |
|   телефона    |     Номер     |наименование уч| номер отдела  |
+---------------+---------------+---------------+---------------+
|       л       |    +7 111     |    Иванов     |  ул Ленина 5  |
+---------------+---------------+---------------+---------------+
|       л       |    +7 222     |    Иванов     |  ул Ленина 1  |
+---------------+---------------+---------------+---------------+
|       л       |    +7 333     |    Петров     |Центральная 10 |
+---------------+---------------+---------------+---------------+
|       с       |    +7 444     | Рога и Копыта |      Б-7      |
+---------------+---------------+---------------+---------------+
После сортировки получился такой справочник:
+---------------+---------------+---------------+---------------+
|      Тип      |               |Фамилия влад./ | адрес влад./  |
|   телефона    |     Номер     |наименование уч| номер отдела  |
+---------------+---------------+---------------+---------------+
|       л       |    +7 333     |    Петров     |Центральная 10 |
+---------------+---------------+---------------+---------------+
|       л       |    +7 222     |    Иванов     |  ул Ленина 1  |
+---------------+---------------+---------------+---------------+
|       л       |    +7 111     |    Иванов     |  ул Ленина 5  |
+---------------+---------------+---------------+---------------+

Доступен запрос "поиск по фамилиям". Введите нужную фамилию:
```

Проверим запросы на 2, 1 и 0 результатов:

```
Доступен запрос "поиск по фамилиям". Введите нужную фамилию:
Иванов

По фамилии "Иванов" найден номер: +7 222
По фамилии "Иванов" найден номер: +7 111
```

```
Доступен запрос "поиск по фамилиям". Введите нужную фамилию:
Петров

По фамилии "Петров" найден номер: +7 333
```

```
Доступен запрос "поиск по фамилиям". Введите нужную фамилию:
Дворников

В справочнике нет номеров, принадлежащих этому человеку
```